

SOFTWARE TEST PLAN

Project Name:	FlightSound
Team:	06

Revision History:

Revision #	Revision Description	Revised Date	Revised By	Approved By
1.0	Initial Draft	15 February 2026	Tiffany Phan	Carter Lim
1.1	Updates	28 April 2026	Carter Lim	Tiffany Phan
1.2	Updates	29 April 2026	Tiffany Phan	Carter Lim

Table of Contents

1. Introduction.....	3
1.1 Purpose.....	3
1.2 Scope.....	3
1.3 References.....	3
2. Test Strategy.....	3
2.1 Testing Objectives.....	3
2.2 Testing Levels.....	3
Unit Testing.....	3
Integration Testing.....	3
System Testing.....	3
User Acceptance Testing (UAT).....	3
Regression Testing.....	4
Smoke Testing.....	4
3. Automated Testing.....	4
3.1 Continuous Integration (CI).....	4
3.2 Unit Testing.....	4

Goal:.....	4
3.3 API and Integration Testing.....	4
3.4 End-to-End Testing.....	4
3.5 Static Code Analysis.....	4
3.6 Performance Testing.....	5
4. Test Environment.....	5
Development Environment.....	5
Testing Environment.....	5
Database.....	5
5. Entry and Exit Criteria.....	5
Entry Criteria:.....	5
Exit Criteria:.....	5
6. Risks and Mitigation.....	6
Risk: Integration failure.....	6
Risk: Data loss.....	6
Risk: Poor performance.....	6
7. Deliverables.....	6
Appendix A — Test Case Template.....	6

1. Introduction

1.1 Purpose

This Software Test Plan (STP) documents the testing performed for the FlightSound project. FlightSound is a web application that displays real-time flight data near airports and estimates noise impact for users.

This document reflects the testing completed by the team. The majority of testing was performed manually, with limited automated testing for frontend authentication features.

1.2 Scope

This test plan covers:

- Airport search and selection
- Flight data retrieval from AirLabs API
- Noise level estimation and display
- Google Maps visualization
- Weather data display via Open-Meteo API
- Google Sign-In authentication
- User preferences (airport and location)

- Session management (localStorage)
- Frontend–backend integration
- SQLite database operations

The following were not fully implemented or tested:

- Backend unit testing using JUnit
- Automated end-to-end testing
- Performance/load testing
- CI/CD pipelines
- Scientific validation of noise accuracy

1.3 References

- Software Requirements Document (SRD)
- Software Design Document (SDD)
- AirLabs API Documentation
- Open-Meteo API Documentation

2. Test Strategy

2.1 Testing Objectives

The main goals of testing were:

- Ensure the application runs without crashing
- Confirm flight data loads correctly from the API
- Verify flights appear on the map and sidebar
- Confirm noise levels are calculated and displayed
- Verify login and session behavior works
- Confirm preferences can be saved and loaded
- Identify bugs before final submission

2.2 Testing Approach

Testing for FlightSound was primarily manual.

The team tested the application by running it locally and interacting with it as a user. This included:

- Running the backend server
- Opening the frontend in a browser
- Searching for airports
- Viewing map markers and flight data

- Checking noise calculations
- Testing login and session behavior
- Saving and loading user preferences

Automated testing was limited to frontend authentication utilities using Jest and Fast Check.

Backend testing, API testing, and system testing were all performed manually.

2.3 Testing Levels

Unit Testing

- **Frontend:** Automated tests exist for authentication logic (auth.test.js). These tests verify session validation, JWT decoding, session storage, and UI behavior.
- **Backend:** Unit testing was not implemented. Backend logic was tested manually by running the application and verifying outputs.

Integration Testing

Integration testing was performed manually to verify:

- Frontend → Backend communication
- Backend → AirLabs API
- Backend → Open-Meteo API
- Backend → SQLite database

System Testing

System testing was performed manually by running the full application:

1. Start backend server
2. Open frontend
3. Search airports
4. View map and sidebar
5. Check flight and noise data
6. Log in
7. Save preferences
8. Reload and verify persistence

User Acceptance Testing (UAT)

Testing was informal. Team members acted as users and verified:

- Noise values appeared reasonable
- Interface is usable

- Login flow works
- Preferences persist

Regression Testing

Performed manually after updates:

- Airport search
- Map display
- Login
- Preferences

Smoke Testing

Basic checks before demos:

- Server runs
- Frontend loads
- Airport search works
- Map displays
- No major errors

3. Automated Testing

3.1 Frontend Testing

Automated tests exist for authentication features using:

- Jest
- Fast-check

These tests verify:

- Session validity
- JWT decoding
- Session storage behavior
- Navbar UI state
- Google account authentication

3.2 Backend Testing

Backend testing was manual only.

Testing methods included:

- Running the server (FlightSoundServer.java)
- Calling endpoints through the frontend
- Observing responses and console output

3.3 API Testing

API testing was performed manually using:

- Browser requests
- Developer tools (Network tab)

The team verified:

- /api/noise returns flight and noise data
- Invalid inputs return errors
- User and preference endpoints function correctly

3.4 End-to-End Testing

End-to-end testing was manual only. The full user flow was tested through the browser.

3.5 Performance Testing

Formal performance testing was not conducted. Performance was observed during normal use. Some API delays were noticed.

4. Test Environment

Development Environment

- Local machines
- Visual Studio Code
- Kiro
- Java backend server
- HTML/CSS/JavaScript frontend
- Chrome browser

Backend Environment

- Java HTTP server (port 8080)

Database

- SQLite (users.db)

5. Features Tested

- Airport search
- Flight data retrieval
- Noise calculation
- Map visualization
- Weather display
- Authentication
- Session persistence
- Preferences

All testing was manual.

6. Entry and Exit Criteria

Entry Criteria

- Feature is implemented
- API keys configured
- Backend runs
- Frontend loads

Exit Criteria

- Feature works during manual testing
- No major errors
- Core functionality demonstrated
- Automated tests (if any) pass

7. Risks and Limitations

Limited Automated Testing

- Most testing is manual.

External API Dependency

- The app depends on live APIs.

Noise Accuracy

- Noise values are estimates.

No Backend Unit Tests

- Backend logic not automatically verified.

Manual Testing Limitations

- May miss edge cases.

8. Deliverables

Limited Automated Testing

- Software Test Plan (STP)
- Manual test cases
- auth.test.js
- Final demo

Appendix A — Test Case Template

Test cases will be created using the following format

Test Case ID	Title	Preconditions	Steps	Expected Result	Actual Result	Status (Pass/Fail)	Severity	Priority	Notes
TC-01: Load Application	Short name	☐ User exists ☐ Test data ready	1) ... 2) ... 3) ...	What should happen	What actually happened	☐ Pass ☐ Fail	Low/Med/High	Low/Med/High	Additional info

TC-FS-001: Display Flight Data on Map

Field	Sample Content
Test Case ID	TC-FS-001
Title	Display Current Flight Data on Map
Preconditions	Backend server running, API keys configured
Steps	1. Open the FlightSound web application 2. Enter airport code (e.g., LAX) 3. View maps
Expected Result	Flight markers appear on the map and flight details display correctly
Actual Result	Flight markers displayed successfully. Aircraft details and noise levels appeared correctly.
Status (Pass/Fail)	Pass
Severity	High
Priority	High
Notes	Minor delay (~1–2 seconds) when loading flight data

TC-FS-002: User Login Session Persistence

Field	Sample Content
Test Case ID	TC-FS-002
Title	Verify login session is saved
Preconditions	User completes Google Sign-In
Steps	1. Click Google Sign-In 2. Complete login 3. Reload page
Expected Result	User remains logged in and session persists
Actual Result	Login persisted and user session remained after reload
Status (Pass/Fail)	Pass
Severity	Medium
Priority	Medium
Notes	Session stored in localStorage

TC-FS-003: Save User Preferences

Field	Sample Content
Test Case ID	TC-FS-003
Title	Save and load user preferences
Preconditions	User is logged in
Steps	1. Enter preferred airport and location (longitude & latitude) 2. Save preferences 3. Reload page
Expected Result	Preferences persist after reload
Actual Result	Preferences successfully saved and reloaded
Status (Pass/Fail)	Pass
Severity	Medium
Priority	Medium
Notes	Data stored in SQLite database

TC-FS-004: Invalid Airport Handling

Field	Sample Content
Test Case ID	TC-FS-004
Title	Handle invalid airport input

Preconditions	Backend server running
Steps	1. Enter invalid airport code (e.g. ZZZZ) 2. Submit search
Expected Result	Error message appears, no crash
Actual Result	"No flights found" message displayed, application remained stable
Status (Pass/Fail)	Pass
Severity	Medium
Priority	Medium
Notes	Input handled safely without breaking UI